Your Voice. Your Stories.

Seven Complete Speakable STAR Answers

Prepared for your interview with Zid

Background used: 8-12 yrs · Financial Services · Network Outage Incident Lead · Full Production Stack

How to use this document:

Each answer is written in the first person as words you can say out loud, right now. Read them aloud at least three times each — not to memorise them word for word, but until the shape of the story feels natural in your mouth. Adapt any detail that doesn't match your real experience. The coach notes explain what Zid is listening for in each section so you understand the purpose of every sentence, not just the words.

> **One rule:** Never say anything in this document that is not true. These are scaffolds built on your real background. If a specific detail does not match your experience, replace it with one that does. Zid will probe every story. Authenticity is not optional — it is the strategy.

QUESTION 1 The Serious Incident

"Tell me about the most serious production incident you have personally led. What was at stake, how did you handle it, and what did you change afterward?"

⏱ Target: 4-5 minutes total. Spend 45 seconds on Situation, 30 seconds on Task, 2.5 minutes on Action, 1 minute on Result.

SITUATION

We were running a trading platform at a mid-sized financial services firm — equities and fixed income desks sharing core Linux infrastructure. One Thursday morning, about forty minutes before market open, I got a page saying our primary market data distribution network had gone silent. Not degraded — completely silent. The trading desks had no live prices feeding into their systems.

That window between 8:20 and the 9:00 open is when traders are positioning for the day. Losing forty minutes of data there is not a minor inconvenience — it directly affects their ability to manage risk going into the open. We had senior traders on the phone within five minutes.

> *Coach note: The detail about the pre-market window matters. It shows Zid you understand the business context of the infrastructure you protect — not just the technical side. He will register that immediately.*

TASK

I was the on-call lead that week. I had two engineers available — one junior who could handle communications and runbook tasks, one more senior who I'd put on parallel investigation. My job was to run the diagnosis, direct the team, and be the single point of communication to the trading desk manager and our infrastructure director who was already asking for updates.

The hardest part of that task structure is that you cannot disappear into the technical problem. You have to keep one eye on the investigation and one eye on the clock and the stakeholders at the same time.

> *Coach note: This is where Zid hears leadership, not just technical skill. He is listening for how you organised the response — not just what you found.*

ACTION

My first check was whether this was a network path issue or a data source issue. I ran a quick ping sweep across the multicast-capable interfaces and found that the servers were reachable — which told me the physical network was not down. But when I checked the multicast group memberships with 'ip maddr show', the servers had dropped their subscriptions to the market data groups.

That narrowed it fast. This wasn't a link failure — it was an IGMP issue. I pulled the switch logs and found that a network engineer had pushed a firmware update to the distribution layer switches overnight. That update had reset the IGMP snooping tables on three VLANs — silently. The switches had stopped forwarding our multicast streams because the membership records had been wiped.

At that point I wasn't completely certain — my instinct said firmware rollout, but I needed to confirm before I started making changes on live switches. I had my junior engineer capture traffic with tcpdump on two of the affected servers simultaneously. He confirmed: frames were hitting

the NIC on the upstream feed server but nothing was arriving downstream. That told me the problem was definitively at the switch forwarding layer.

I called the network team, walked them through what I was seeing, and we agreed on the remediation: re-enable IGMP snooping querier on the affected VLANs and force a membership refresh. The network engineer made the changes, I watched the multicast group memberships come back on the servers in real time, and I had my senior engineer verify end-to-end data flow before I called it resolved.

Total time from my first page to full data restoration: twenty-two minutes. The market data was flowing eleven minutes before open.

> *Coach note: The moment of uncertainty — 'I wasn't completely certain' — is important. Real stories have it. It also shows diagnostic rigour: you confirmed before acting rather than guessing on live infrastructure. Zid will respect that.*

RESULT

The trading desks had full data before open. We did a post-mortem the following day. The root cause was straightforward — a network change without a cross-team notification to infrastructure. But the more important finding was that we had no monitoring alert for multicast group membership drops. We only knew about the problem because a trader called us.

I wrote a Python script that ran every sixty seconds, checked the expected multicast group memberships on every market data server against a config file, and fired a Prometheus alert if any group was missing. We tested it, deployed it, and within the next three months it caught two smaller IGMP disruptions automatically — before any trader noticed.

We also added a mandatory cross-team notification requirement for any network change touching the market data VLANs. Simple process change, but it closed the gap that caused the incident in the first place.

> *Coach note: Ending with two concrete permanent changes — the monitoring script and the process change — is what separates a senior engineer's answer from a junior one. Zid will remember this ending more than anything else in the story.*

QUESTION 2 Pushing Back on Someone Senior

"Tell me about a time you had to push back on a change or a request from someone more senior than you. How did you handle it?"

⏱ Target: 3-4 minutes. The Action section is the most important here — Zid wants to hear HOW you did it.

SITUATION

About two years into my current role, we had a situation where a senior VP on the trading technology side wanted us to push a configuration change to our high-availability cluster during market hours. The change was to a network bonding configuration on the servers that handled order routing — the idea was to switch from active-backup bonding to LACP to improve throughput.

The request came in at around 11am on a Tuesday with a request to implement it that afternoon. The VP's reasoning was that they had a throughput bottleneck showing up in their metrics and they wanted it resolved before end of week.

TASK

My concern was specific and technical. Switching bonding modes on a live system requires bringing the bond interface down briefly. On our cluster configuration, that would cause a failover event — which our monitoring would flag as a partial outage. We had tested the failover in staging and it took between eight and fifteen seconds depending on which node initiated it. During those fifteen seconds, new order routing connections would be dropped.

During active trading hours, fifteen seconds of dropped connections on an order routing server is not acceptable. The VP didn't have visibility into that dependency.

ACTION

I didn't just say no. I sent him a short written summary — three paragraphs — explaining specifically what would happen during the change, what the risk window was, and what I recommended instead. Not a complaint. A technical brief with a proposed alternative.

The alternative I proposed was to do the change in our next scheduled maintenance window which was four days away — Thursday night after 18:00. I also offered to demonstrate the exact failover behaviour in our staging environment that afternoon so he could see what his traders would experience if we did it live.

He took me up on the staging demo. We ran it, he watched the connection drop counter, and he understood immediately why the timing mattered. He agreed to the maintenance window without any friction. In fact he appreciated that I'd given him something concrete to look at rather than just telling him it was risky.

> *Coach note: The staging demo offer is the key move here. It turned an abstract technical concern into something the VP could see with his own eyes. It also showed respect — you didn't lecture him, you gave him evidence and let him reach the conclusion himself. Zid will love this answer.*

RESULT

We made the change in the maintenance window with no issues. The throughput improvement he was looking for was confirmed the following morning. And from that point on, that VP always ran significant network changes past me before scheduling them — which made my life and his considerably easier.

The broader lesson I took from it was that most push-back situations are not confrontations — they're information gaps. He didn't know what the change would do to the cluster. Once he did, the decision was obvious. My job was to surface the information clearly, not to win an argument.

> *Coach note: That last line — 'my job was to surface the information, not win an argument' — is the philosophical close Zid is looking for. It shows maturity and systems thinking, not ego.*

QUESTION 3 Learning Something New Under Pressure

"Describe a time when you had to learn something completely new under time pressure to solve a production problem. How did you approach it?"

⏱ Target: 3-4 minutes. The Action section should show your learning methodology — specific, disciplined, not random.

SITUATION

A few years ago we migrated from a legacy monitoring stack to Prometheus and Grafana. I had strong operational Linux knowledge but I had not worked with Prometheus at depth — I understood the concept but I'd never written PromQL queries or designed an alerting rule set from scratch for a financial services environment.

The situation that forced me to learn fast was a performance regression on one of our HA clusters that our existing monitoring had completely missed. We had tools that told us whether services were up or down, but nothing that gave us granular visibility into CPU scheduling latency, network queue depth, or kernel memory pressure. We were essentially flying blind on the metrics that matter most for trading workloads.

This became urgent when we had two incidents in the same month that we only found out about because traders called us. After the second one, my manager made it clear that we needed proper observability within thirty days.

TASK

I owned the implementation. No external consultants, no vendor support contract. A team of five, a thirty-day window, and a production environment I couldn't afford to instrument incorrectly.

ACTION

I didn't start by reading the Prometheus documentation from the beginning. That's too slow and too unfocused under time pressure. Instead I started with the specific questions I needed the system to answer — what metrics do I need to detect the exact type of incident we just had? I wrote those questions down as eight concrete statements: 'I need to know when NIC receive drop rate exceeds X per second on any market data interface.' 'I need to know when iowait on storage nodes exceeds Y percent sustained for more than thirty seconds.' That level of specificity.

Then I reverse-engineered from those requirements into the tooling. I stood up a Prometheus instance in our staging environment within the first two days and started exporting node_exporter metrics from our cluster. I deliberately used production-equivalent load in staging — replaying real traffic patterns — so the dashboards I built would reflect real behaviour, not synthetic test conditions.

The part I found genuinely difficult was PromQL — the query language is powerful but non-obvious for someone coming from a shell scripting background. I spent three evenings reading the Prometheus operator documentation and the Brian Brazil book, specifically focused on rate functions and histogram quantiles. I also reached out to a colleague at another firm who had been running Prometheus for two years and asked him one targeted question: what are the three PromQL mistakes that cost you the most time to fix? His answer saved me probably a week of trial and error.

Within three weeks I had a complete dashboard covering CPU scheduling latency, NIC drops, disk IO wait, memory reclaim events, and process-level resource consumption — all with alerting rules tuned to thresholds that had meaning for our specific workload.

> *Coach note: The detail about asking the colleague one targeted question is important — it shows you know how to use other people's knowledge efficiently rather than either ignoring it or over-relying on it. Senior engineers know how to extract specific value from a network.*

RESULT

We went live on production within the thirty-day window. In the first two months after deployment, the monitoring caught three issues before any trader noticed — including one that turned out to be early signs of a failing NIC, which we replaced before it caused an outage.

I documented the entire implementation — architecture decisions, PromQL patterns, alerting philosophy — as an internal runbook. That documentation is now the reference for anyone on the team who needs to add new metrics or modify alerting rules. The knowledge didn't stay with me — it became part of the team.

QUESTION 4 Improving Something Unprompted

"Tell me about a time you improved a system or process that nobody had asked you to improve. What drove you to do it, and what was the result?"

⏱ Target: 3 minutes. Keep Situation and Task tight — the payoff is in Result.

SITUATION

About eighteen months into a role I noticed that our Ansible playbooks — we had about sixty of them at that point — had been written by three different engineers over four years and had no consistent structure, no standard error handling, and very inconsistent variable naming conventions. They worked, in the sense that they executed without failing. But they were brittle in ways that weren't obvious until something broke at 2am.

Nobody had flagged this as a problem. There were no tickets about it. The playbooks ran in CI, they deployed to production, and the team moved on. But every time I had to modify one — particularly playbooks I hadn't written — I spent time just understanding the structure before I could safely change anything. That friction was invisible in any metric but it was real and it was costing us.

TASK

I made a decision to spend two weeks, in parallel with my regular work, to refactor the most-used twenty playbooks into a consistent structure and write a standards document that would govern any new playbooks going forward. This was entirely self-directed — I didn't ask for permission because I didn't need it, but I did let my manager know what I was planning before I started.

ACTION

I started by identifying which playbooks had been run most frequently in the last six months using our AWX job history. The top twenty accounted for about eighty percent of all automation runs — that's where the effort would have the highest impact.

For each one I documented what it actually did versus what its name suggested it did — in two cases those were different things, which was its own small discovery. I refactored them into a consistent role-based structure with standardised variable naming, idempotency checks, and explicit failure handlers that wrote meaningful error messages rather than just exiting with a generic error code.

I also introduced ansible-lint into our CI pipeline so that new playbooks that violated the standards would fail the build before they ever got reviewed. The standards enforced themselves automatically from that point forward.

I ran the refactored playbooks in staging against full production-equivalent inventory for a week before proposing to merge them. I wanted to be certain that my tidying hadn't changed any behaviour, only structure.

RESULT

The refactored playbooks went live without incident. The more measurable outcome came over the following year: our mean time to make a safe change to an Ansible playbook dropped significantly — I tracked it informally and it went from typically forty-five minutes of reading and verifying to about fifteen for playbooks that followed the new standard.

The more important outcome was one I didn't expect. A junior engineer who joined six months later told me the documentation and structure made it possible for him to write his first production playbook independently, without asking for help, within his first month. That was more satisfying than any metric I could have reported.

> *Coach note: That last sentence about the junior engineer is one of the best things you can say to Zid. It shows you think about knowledge transfer and team capability, not just your own output. He is building a team for the long term — he needs people who make the whole team better.*

QUESTION 5 On-Call and the Follow-the-Sun Model

"How do you manage being on a global follow-the-sun on-call rotation? Tell me about the most challenging on-call experience you've had."

⏱ Target: 3-4 minutes. Be honest about the difficulty — Zid knows what on-call actually is. Don't sanitise it.

SITUATION

On-call in financial services is not optional and it is not always convenient. I've been on rotations where we covered a follow-the-sun model across three timezones, and I've been the sole on-call engineer for a two-week stretch during a colleague's illness. I want to be honest about what that involves before I tell you the specific story, because I think it's important you know I understand what I'm signing up for — not just in theory.

The most challenging on-call experience I had was a Saturday morning page at 5:47am. We had a network segmentation issue that had developed overnight — a routing change that had been applied by an automated provisioning script had misconfigured the VLAN tagging on a set of storage interfaces. By the time I was paged, three of our HA cluster nodes had lost connectivity to the shared storage. The cluster had partially failed over but the failover itself had created a split-brain condition.

TASK

Split-brain in a financial services HA cluster is one of the most dangerous situations you can be in. Both nodes believe they are the primary. Both nodes may be trying to write to the same storage. The risk of data corruption is real. My immediate priority was not to restore service — it was to prevent the situation from getting worse while I understood what state everything was in.

ACTION

The first thing I did — before touching anything — was take a full snapshot of the current state. I ran 'crm_mon -1' to get the cluster status, checked the STONITH fencing logs to understand what the cluster had already attempted, and verified which node had the storage connections and which did not. I needed to know exactly what the cluster thought before I told it what to do.

The split-brain was real — both nodes had promoted themselves. I had to make a deliberate decision about which node to trust. I chose the node that had maintained its storage connectivity and had the more recent cluster epoch. I manually fenced the other node using STONITH — which means I deliberately powered it off — to force the cluster into a clean single-primary state.

Fencing a production node on a Saturday morning is not a small decision. I documented my reasoning in a running incident note in real time as I worked — partly for the post-mortem, partly because I wanted a clear record that my decisions were deliberate and justified, not panicked.

Once the cluster was in a clean state I investigated the root cause. The automated provisioning script had a bug — when it processed the VLAN configuration change, it applied the new tag to the data interfaces but not to the storage heartbeat interfaces. The heartbeat loss triggered the split-brain. I corrected the VLAN tags manually on the fenced node, brought it back into the cluster in standby mode, and watched the resync complete cleanly before I declared the incident resolved.

Total time: two hours and fourteen minutes. I stayed on comms with my manager the whole time — brief updates every twenty minutes, even when I had nothing new to report, just 'still investigating, no further degradation.'

> *Coach note: 'Brief updates every twenty minutes even when I had nothing new to report' — this is what Zid wants to hear. Senior stakeholders in a crisis do not want silence. They want cadence. This shows you understand the communication discipline of incident management, not just the technical discipline.*

RESULT

Full cluster restored by 8am — before any trading activity. The post-mortem identified two changes: we added a pre-flight check to the provisioning script that validated storage interface VLAN config before applying any network changes, and we added a specific monitor for storage heartbeat interface state that would have caught this before the split-brain occurred.

On the personal sustainability side — I've learned over the years that good on-call is mostly the result of good engineering done before the page arrives. Every runbook I write, every alert I tune, every failure mode I document in advance is reducing the cognitive load of whoever is woken up at 5:47am. I invest in that work specifically because I know what it's like to be the one woken up.

> *Coach note: That closing reflection is important and genuine. Zid has been running on-call infrastructure for twenty years. He will recognise that philosophy immediately — it is exactly how experienced engineers think about operational work.*

QUESTION 6 Technical Disagreement with a Colleague

"Tell me about a time you had a significant disagreement with a colleague about the right technical approach. How did you resolve it?"

⏱ Target: 3 minutes. Show you argue from evidence, not ego — and that you can update your view gracefully.

SITUATION

A couple of years ago my team was designing the monitoring and alerting strategy for a new set of Linux servers being brought into the trading environment. A senior colleague — someone I respected and had worked closely with for two years — proposed that we set our alert thresholds based on absolute values: CPU above 80 percent, memory above 75 percent, that kind of approach.

My position was that static absolute thresholds are the wrong model for trading infrastructure. A server running a market data feed handler at 60 percent CPU during normal market conditions and a server running a batch reconciliation job at 60 percent CPU are not in the same situation. The same number means something completely different in context.

TASK

We needed to agree on an approach before deploying the monitoring stack, because changing alerting philosophy after the fact means retuning dozens of rules and re-baselining everything. Getting this right at the design stage mattered.

ACTION

We had a direct conversation about it — not a heated argument, but a genuine technical discussion. I laid out my concern specifically: static thresholds in trading environments produce two failure modes. Either they're set conservatively and you get alert fatigue from normal operational variance during market events, or they're set loosely and you miss the early signal of a real problem because the threshold isn't crossed until things are already bad.

He pushed back reasonably. His concern was that dynamic baseline alerting — alerting based on deviation from historical normal — is more complex to configure, harder to explain to stakeholders, and can mask genuine degradation if the baseline itself drifts gradually.

He was right about that risk. I had to acknowledge it. Gradual baseline drift is a real failure mode in adaptive alerting and I hadn't addressed it in my initial proposal.

We agreed to test both approaches in parallel in staging over a two-week period using replayed production traffic. We each wrote our alerting rules, ran them against the same data, and compared the false positive and false negative rates. The results were reasonably clear: my approach produced fewer false positives but required an additional check for baseline drift. His approach was simpler but would have generated significant alert noise during the specific market volatility events in our test data.

We landed on a hybrid: static thresholds for the most critical metrics where any deviation is genuinely significant, and baseline-relative alerting for the noisier operational metrics — with a weekly automated check for baseline drift that would flag if any metric's normal range had shifted more than fifteen percent over thirty days.

RESULT

We deployed the hybrid model. It's been running for over a year and the false positive rate is low enough that the team actually responds to alerts rather than filtering them. That's the real measure of a good alerting strategy — whether people trust the alerts.

The more important outcome for me personally: my colleague's concern about baseline drift turned out to be genuinely valid. We caught one case of gradual drift in the first six months that would have hidden a real memory pressure issue. I was wrong to dismiss that risk initially and I told him so. He's one of the engineers I learn most from.

> *Coach note: Saying out loud that a colleague was right and you were wrong, and that you told them so — this is rare in interviews. Zid will find it refreshing. It signals intellectual honesty and professional maturity, which are exactly the qualities he needs in someone who will operate with significant autonomy on his team.*

QUESTION 7 Why Morgan Stanley and Why This Team

"Why Morgan Stanley specifically, and why this team? What does low-latency infrastructure mean to you beyond the technical challenge?"

⏱ Target: 2 minutes maximum. This is not a technical answer — it is a values answer. Be direct and genuine. Do not over-explain.

THE ANSWER — say this naturally, in your own rhythm

I've spent roughly a decade in financial services infrastructure — and the honest answer is that I've been moving toward this kind of role for most of that time without always having a precise name for what I was moving toward.

What I've found, working on trading systems specifically, is that the infrastructure is not a support function for the business — it is the business. When the network drops a packet, the trade doesn't happen. When the timestamp is wrong by two milliseconds, you have a compliance problem. When the cluster fails over in fifteen seconds instead of eight, those seven seconds have a measurable cost. I find that accountability genuinely motivating. Not stressful — motivating. I prefer environments where what I do has a direct, measurable consequence.

What draws me to Morgan Stanley specifically, and to this team, is the combination of scale and depth. This isn't a firm that is experimenting with low-latency infrastructure — it has been running and refining it for decades. The problems here are not about whether to use kernel bypass — they're about how to squeeze the next fifty nanoseconds out of a system that's already extremely well-engineered. That's the kind of problem I want to work on.

And the fact that this team has been led by someone who has been here since 2006 tells me something specific: that the people here find the work worth staying for. In an industry where experienced Linux engineers can move anywhere, that kind of tenure means something. It tells me this is a place that invests in its people and takes the craft seriously. That matters to me when I'm thinking about where I want to spend the next ten years.

↯ After you finish — stop. Let him respond. Do not add a qualifier or a question. The silence is intentional.

> *Coach note: The reference to his tenure without naming him directly — 'led by someone who has been here since 2006' — is respectful and shows you've done your research without it feeling like flattery. He will know you're talking about him and appreciate that you noticed without making it awkward.*

PRACTICE GUIDE — HOW TO PREPARE WITH THIS DOCUMENT

Reading these answers is not preparation. Speaking them is. Here is a structured practice method that will make these stories feel genuinely yours by the day of the interview.

Days 1-2: Read and internalise

Read each answer slowly out loud, once, all the way through. Do not try to memorise. You are absorbing the shape of each story — where it starts, what the turning point is, how it ends. After reading each one, close the document and summarise the story back to yourself in three sentences. If you can't, read it again.

Days 3-4: Personalise any detail that doesn't fit

Go through each answer and flag any specific detail that doesn't match your real experience. Replace it with something true. The technology names, the timings, the team sizes — all of these can change. The structure and the emotional truth of the story should stay. Write your personalised version in a separate document in your own words.

Days 5-6: Speak without reading

Set a timer for each answer — the target times are shown at the top of each question. Tell the story without reading from the document. Record yourself on your phone if possible. Listen back and check: does it sound like you're remembering something that happened, or reciting something you wrote? If it sounds recited, tell it again more conversationally — accept that you'll use slightly different words each time. That's correct. That's how real stories sound.

The night before: one final read

Read the one-page summary from the Round 3 preparation document. Then read the opening of each answer — just the first sentence of each Situation section. That's all you need to trigger the story. Sleep. The preparation is done.

These are your stories. You lived them.

Walk into that room and tell them the way only you can.